Orca 중앙 오케스트레이션 체제

여러 Unity 프로젝트를 **하나의 규칙 베이스**로 운용하는 구조를 구축했다. 메인이 명령을 받아 분류하고, 대상 이름으로 에이전트를 배정하고, 작업 결과를 그 이름으로 축적한다 — **다섯 단계가 모두 실제로 동작**하며, 그 위에 병렬 실행과 외부 검증을 얹을 설계까지 마쳐둔 상태다.

2026-08-26 · D:\Orca · 에이전트 10 · 누적 기록 140건 · 자동 훅 3

---

0. Orca가 무엇인가

Orca는 AI 에이전트를 격리해서 돌리고, 그 결과를 비교해 채택하는 도구다. 작업 하나마다 git worktree(같은 저장소의 독립된 작업 폴더)와 브랜치를 만들어 CLI 에이전트를 붙이고, 나온 변경을 diff로 견줘 쓸 것만 합류시킨다. 에이전트를 많이 켜서 빨라지는 도구가 아니라, **여러 시도를 비교 가능한 형태로 바꿔주는 도구**다.

이 문서가 다루는 **Orca 워크스페이스**는 그 위에 얹은 운용 베이스다. 게임 코드는 각 Unity 프로젝트에 있고, 여기엔 **코딩 규칙 · 에이전트 정의 · 프로젝트별 작업 기록 · 실행 스크립트**만 모았다. 어느 프로젝트에서 세션을 열든 같은 규칙과 같은 에이전트 구성으로 시작하게 만드는 것이 목적이다.

구성	역할
규칙 문서 4종	코딩 규칙 · 프리팹 규칙 · 에이전트 라우팅 기준 · 행동 지침. 모든 세션이 자동으로 읽는다.
에이전트 정의 10	설계 · 구현 · 버그 수정 · QA · 커밋 검토 등 역할별 전문 에이전트. 각자 담당 범위와 모델이 고정.
작업 기록 140건	클래스 · 프리팹별로 무엇을 왜 어떻게 고쳤는지 누적. 다음 작업의 출발점.
자동 실행 훅 3	세션 시작 시 에이전트 배치와 프로젝트 목록 제시, 매 턴 확인, 종료 시 정리.

1. 명령이 지나가는 길 — 5단계 전부 가동 중

단계	하는 일	구현	근거 문서
1 수신	**공통 베이스 로드**	루트 `CLAUDE.md`가 CODE / PREFAB / AGENT를 함께 로드. 규칙 정본은 워크스페이스에 1부.	CLAUDE.md
2 분류	**축 판정 → 에이전트 선택**	설계 / 컨텐츠 / 분량(대·소) / QA(대·소) / 마무리. 축마다 model·effort가 정의에 고정돼 있다.	AGENT.MD
3 배정	**이름 기반 소프트 락**	`{Project}_{Class}_class`로 스폰. 같은 대상이 작업 중이면 자동으로 순위를 미룬다.	ParallelWorkContract §11

단계	하는 일	구현	근거 문서
4 작업	**소유 파일만 수정**	작업 전 대응 md를 먼저 읽는다. 동작이 바뀌는 수정은 `work/{날짜}-{주제}` 브랜치에서.	WorkBranchFlow.MD
5 학습	**이름 단위 축적**	`.claude/class/{이름}.md` · `prefab/{이름}.md`로 저장. 요청 없어도 항상 수행된다.	CLAUDE.md

3단계 이름에 프로젝트명을 앞에 붙인 건 실측으로 얻은 설계다 — 두 프로젝트가 같은 공용 라이브러리를 써서 클래스명이 겹치는데, 접두사가 이 오인 충돌을 정확히 갈라준다(2026-08-21 레인 3개로 확인).

2. 지향 대비 현황

지향	현황	내용
기본 베이스 통일	가동	규칙 정본 1부로 통일. PC마다 다른 경로는 `LocalPaths.json`으로 분리해 어느 장비에서도 같은 상태로 이어진다.
메인이 분류 → 에이전트 선택	가동	별도 감독 계층 없이 메인이 직접 라우팅. 축이 겹치면 더 무거운 쪽으로 올리는 판정 기준까지 명문화.
Class / Prefab 이름으로 작업	가동	명명 규칙 + 사전 조회 + 대기 프로토콜이 한 세트로 동작. 실제 레인 실행으로 검증 완료.
작업 결과를 학습으로 축적	가동	class 109 · prefab 24 · design 5 · qa 2 축적. 다음 작업이 이 기록에서 출발한다.
동일 대상이면 순위를 뒤로	가동	활성 여부를 조회해 대기 후 진입. 프로젝트 접두사 덕에 동명 클래스라도 서로 다른 프로젝트면 동시 진행된다.
설계·컨텐츠 병렬 분배	설계 완료	수직(도메인) 분할, 메인 선점 후 커밋, 레인별 소유 파일·ID 대역 계약서까지 규격이 서 있다. 레인 2개 실측만 남았다.
외부 모델(Codex) 교차검증	확장 예정	붙일 지점이 이미 두 곳 뚫려 있고, 커밋 직전 검토 단계가 그 자리를 먼저 채우고 있다.
간단한 작업은 로컬 LLM	확장 예정	모델 계층(opus / sonnet / haiku)이 이미 라우팅에 들어가 있어, 아래 단계를 하나 더 얹는 형태로 확장된다.

3. 이 구조가 선 근거

- 한 Unity 프로젝트에는 **에디터 1 · MCP 브릿지 1 · Play Mode 1 · OS 포커스 1**뿐이다. 중앙 체제는 취향이 아니라 이 제약에서 나온 답이다.
- 그래서 **작성은 병렬, 검증은 중앙 집중**으로 갈랐다. 에디터 락은 검증 시점에만 걸리므로 작업량의 대부분인 작성 구간은 그대로 병렬로 열린다.

- 자르는 축을 파일 종류가 아니라 **컨텐츠 도메인**으로 잡았다. 스키마와 그걸 읽는 코드가 한 레인 안에 통째로 들어온다.
- 충돌은 경고가 아니라 **배분**으로 막는다. 공유 등록 파일은 메인이 미리 선점해 커밋한 뒤 계약서에 적어 넘긴다 — 워커 손에 충돌할 파일 자체가 쥐어지지 않는다.

> **미리 잡아둔 함정 하나:** 워크트리 안에서 검증 명령을 부르면 실행 중인 에디터가 열고 있는 원본이 검증된다. 병렬을 켜기 전에 막을 지점으로 문서에 명시해뒀다.

---

4. 다음 확장

외부 모델 교차검증 — 자리는 이미 준비돼 있다. ① 커밋 직전 게이트: 서브에이전트는 각자 새 컨텍스트라 서로의 실제 시그니처를 못 보고 코드를 쓰는데, 여기에 다른 모델의 독립 판정을 겹치면 같은 맹점을 공유하지 않는다. ② 경쟁 레인의 diff 판독. 두 지점 모두 **종료 코드로 판정하는 검증 명령이 함께 서면 곧바로 값어치가 난다.**

로컬 LLM 결합 — 토큰 절약 방향은 타당하다. 다만 꽂는 위치는 서브에이전트가 아니라 **훅 / CLI 파이프라인**이다(이미 세션 시작 · 매 프롬프트 · 세션 종료 세 지점에서 스크립트가 돌고 있어 그대로 확장 가능). 판단이 아닌 **생성·변환** 구간부터 옮기면 효과가 바로 나온다.

- **커밋 메시지 · 작업 요약** — 짧고 diff로 즉시 대조되니 가장 먼저 옮길 후보.
- **class / prefab 기록 md 초안** — 포맷이 고정이라 로컬 생성에 적합. 확정만 중앙이 맡는다.
- **규칙 위반 스캔** — 규칙이 표로 명시돼 있어 정규식·린터와 조합하면 더 싸게 끝난다.

---

5. 로드맵

순서	할 일	얻는 것
STEP 1	**verify-compile / verify-tables를 CLI 명령으로**	병렬 착수 조건 · 외부 모델 판정 · 로컬 LLM 산출물 검수가 한 번에 열린다.
STEP 2	**커밋 게이트에 두 번째 눈 추가**	서로 다른 모델의 독립 판정으로 맹점을 분산. 산출물은 "고칠 파일:줄" 목록.
STEP 3	**로컬 LLM을 훅 파이프라인에**	정형 작업이 로컬로 내려가며 장기 토큰 비용이 줄어든다.
STEP 4	**레인 2개로 병렬 계약 실측**	이미 완성된 분할 계약을 실제 실행으로 확정한다.

근거: 루트 규칙 문서 4종, ParallelWorkContract.MD, README.md, Unity\README.md, .claude\settings.json, agents\ · hooks\ 실제 파일, 프로젝트별 .claude 기록 집계